

Assignment 4: The Tetris Autoplayer Challenge

Developing an AI Strategy

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Presentation Overview

Objectives

To introduce Assignment 4: developing an Autoplayer (AI) for the game of Tetris. We will cover the project workflow, game mechanics, the grading criteria, and strategies for implementation, connecting these challenges to the algorithmic concepts covered previous weeks.

- 1 The Challenge: Assignment 4 Introduction
- 2 Workflow and Evaluation
- 3 The Codebase and Game Mechanics
- 4 Implementation Guidelines
- 5 Strategy Development
- 6 Conclusion and Advice

The Challenge: Assignment 4 Introduction

The Challenge: Assignment 4 Introduction

Assignment 4: Tetris AI

The Goal

Develop an autoplayer (a simple AI) to play a variant of Tetris and maximize the score.

The Core Task

- You are provided with a Python boilerplate project containing the game engine.
- Your task is to modify the `player.py` file.
- Implement a decision-making algorithm to replace the existing "stupid random auto player".

The Context (Weeks 4 & 5)

This assignment is a practical application of Search Algorithms and Heuristics in a dynamic, resource-constrained environment.

Assignment 4: Tetris AI

The Goal

Develop an autoplayer (a simple AI) to play a variant of Tetris and maximize the score.

The Core Task

- You are provided with a Python boilerplate project containing the game engine.
- Your task is to modify the `player.py` file.
- Implement a decision-making algorithm to replace the existing "stupid random auto player".

The Context (Weeks 4 & 5)

This assignment is a practical application of Search Algorithms and Heuristics in a dynamic, resource-constrained environment.

Assignment 4: Tetris AI

The Goal

Develop an autoplayer (a simple AI) to play a variant of Tetris and maximize the score.

The Core Task

- You are provided with a Python boilerplate project containing the game engine.
- Your task is to modify the `player.py` file.
- Implement a decision-making algorithm to replace the existing "stupid random auto player".

The Context (Weeks 4 & 5)

This assignment is a practical application of Search Algorithms and Heuristics in a dynamic, resource-constrained environment.

Workflow and Evaluation

Workflow and Evaluation

The Development Workflow

The assignment workflow is tightly integrated with Git and an automated grading system.

The Git Process (Fork and Clone)

- 1 **Account:** Create an account on the course GitLab server.
- 2 **Fork:** Fork the master "game boilerplate code" repository.
- 3 **Clone:** Clone your personal repository locally.
- 4 **Develop:** Modify `player.py` and test locally.
- 5 **Submit:** Commit changes and `git push` back to your master repository on GitLab.

The Autograder Mechanics

A `git push` automatically triggers the autograder.

How Grading Works

- 1 **Multiple Runs:** The autograder runs your code multiple times.
- 2 **Random Seeding:** Each run uses a different random seed for the sequence of blocks (tetrominoes).
- 3 **The Metric:** The system calculates the **median score** across all runs.
- 4 **Feedback:** The median score is reported back for that commit.

Submission Strategy Advice

Avoid pushing every minor change, especially near the deadline. The autograder queue gets busy, increasing latency. Push only when you achieve a "substantial improvement".

The Autograder Mechanics

A `git push` automatically triggers the autograder.

How Grading Works

- 1 **Multiple Runs:** The autograder runs your code multiple times.
- 2 **Random Seeding:** Each run uses a different random seed for the sequence of blocks (tetrominoes).
- 3 **The Metric:** The system calculates the **median score** across all runs.
- 4 **Feedback:** The median score is reported back for that commit.

Submission Strategy Advice

Avoid pushing every minor change, especially near the deadline. The autograder queue gets busy, increasing latency. Push only when you achieve a "substantial improvement".

Autograder Constraints (Hard Limits)

The strategy of creating an AI that simply *never loses* (infinite play) is not a viable option. The autograder enforces strict limits.

Resource Limits

Memory: 256 MB RAM limit. Exceeding this terminates the process (no score).

CPU: 50% of one CPU core.

Execution Limits

Time: 15 minutes maximum run time.

Blocks: 400 falling blocks maximum.

If these limits are reached, the game is stopped and the score at that time is recorded.

Autograder Constraints (Hard Limits)

The strategy of creating an AI that simply *never loses* (infinite play) is not a viable option. The autograder enforces strict limits.

Resource Limits

Memory: 256 MB RAM limit. Exceeding this terminates the process (no score).

CPU: 50% of one CPU core.

Execution Limits

Time: 15 minutes maximum run time.

Blocks: 400 falling blocks maximum.

If these limits are reached, the game is stopped and the score at that time is recorded.

Autograder Constraints (Hard Limits)

Resource Limits

Memory: 256 MB RAM limit. Exceeding this terminates the process (no score).

CPU: 50% of one CPU core.

Execution Limits

Time: 15 minutes maximum run time.

Blocks: 400 falling blocks maximum.

If these limits are reached, the game is stopped and the score at that time is recorded.

Implication for Strategy (Week 3: Complexity)

Your algorithm must be efficient. Complex algorithms with high time or space complexity (e.g., excessively deep searches) may hit these limits.

The Codebase and Game Mechanics

The Codebase and Game Mechanics

The Provided Codebase

The cloned repository contains the full game engine and multiple front-end views.

The Views (Front-ends)

Tkinter (`visual.py`): A visual GUI, but "not particularly performant".

Pygame: "Much quicker". Recommended for faster testing of an efficient AI.

Command Line: Text-only interface. Sufficient for development.

Manual Mode

You can play manually using the `-m` flag (e.g., `python visual.py -m`) to understand the dynamics.

- Controls: z/x (rotate), arrows (translate), space (drop).

The Provided Codebase

The cloned repository contains the full game engine and multiple front-end views.

The Views (Front-ends)

Tkinter (`visual.py`): A visual GUI, but "not particularly performant".

Pygame: "Much quicker". Recommended for faster testing of an efficient AI.

Command Line: Text-only interface. Sufficient for development.

Manual Mode

You can play manually using the `-m` flag (e.g., `python visual.py -m`) to understand the dynamics.

- Controls: z/x (rotate), arrows (translate), space (drop).

Game Mechanics: Scoring

The objective is to maximize the score by eliminating rows. The scoring system is highly non-linear.

Scoring Rules

- 1 row: 25 points
- 2 rows (at once): 100 points
- 3 rows (at once): 400 points
- 4 rows (at once, "Tetris"): 1600 points

Strategic Implication

The scoring heavily incentivizes multi-row clears. A strategy focused only on survival (clearing single rows) will score poorly compared to a strategy that builds up for a "Tetris".

Game Mechanics: Scoring

The objective is to maximize the score by eliminating rows. The scoring system is highly non-linear.

Scoring Rules

- 1 row: 25 points
- 2 rows (at once): 100 points
- 3 rows (at once): 400 points
- 4 rows (at once, "Tetris"): 1600 points

Strategic Implication

The scoring heavily incentivizes multi-row clears. A strategy focused only on survival (clearing single rows) will score poorly compared to a strategy that builds up for a "Tetris".

New Features: Bombs and Discards

Two new features have been added to allow for advanced strategies.

The Features

Discard (d key): Allows the player to discard the current falling block (Limited uses).

Bomb (b key): Turns the *next* block into a bomb. When placed, it "blows up all immediately surrounding blocks" and causes blocks above to fall (Limited uses).

Instructor Advice

Personal advice: **ignore these features initially**. Focus on implementing a solid, "regular Tetris" player first. Explore these only after developing a reasonable-scoring AI.

New Features: Bombs and Discards

Two new features have been added to allow for advanced strategies.

The Features

Discard (d key): Allows the player to discard the current falling block (Limited uses).

Bomb (b key): Turns the *next* block into a bomb. When placed, it "blows up all immediately surrounding blocks" and causes blocks above to fall (Limited uses).

Instructor Advice

Personal advice: **ignore these features initially**. Focus on implementing a solid, "regular Tetris" player first. Explore these only after developing a reasonable-scoring AI.

Implementation Guidelines

Implementation Guidelines

The Core Task: `player.py`

Your modifications should be focused within the `player.py` file.

The Structure

- Base Class: `Player`.
- Template: `RandomPlayer` (provided as an example).
- Your Task: Create your own class (e.g., `MyTetrisAI`) by subclassing `Player` or copying `RandomPlayer`.

The Core Task: `player.py`

Your modifications should be focused within the `player.py` file.

The Selector

You must update the global `selected_player` variable to point to your new class.

```
1 # Change this:
2 # selected_player = RandomPlayer
3 # To this:
4 selected_player = MyTetrisAI
```

The Core Function: `choose_action()`

The game engine repeatedly calls the `choose_action(board)` method on the `selected_player` object.

The Responsibility

This method must analyse the current game state (the `board` object) and return the AI's next move (or sequence of moves).

The `RandomPlayer` Example

The `RandomPlayer` simply returns a random choice from the available actions: `left`, `right`, `down`, `anti-clockwise`, `clockwise`.
Your goal is to make this choice intelligent.

The Core Function: `choose_action()`

The game engine repeatedly calls the `choose_action(board)` method on the `selected_player` object.

The Responsibility

This method must analyse the current game state (the `board` object) and return the AI's next move (or sequence of moves).

The RandomPlayer Example

The `RandomPlayer` simply returns a random choice from the available actions: `left`, `right`, `down`, `anti-clockwise`, `clockwise`.
Your goal is to make this choice intelligent.

Accessing the Game State

To make an intelligent decision, the AI must inspect the board object passed to `choose_action()`.

What NOT to do

Using `print(board)` is unhelpful. You must inspect the attributes of the board class (defined in `board.py`).

The Fundamental Data Structure: `board.cells`

The most critical attribute is `board.cells`.

- It is a Python **set**.
- It contains `(x, y)` coordinate tuples for all cells currently occupied by landed blocks on the grid.

Accessing the Game State

To make an intelligent decision, the AI must inspect the board object passed to `choose_action()`.

What NOT to do

Using `print(board)` is unhelpful. You must inspect the attributes of the board class (defined in `board.py`).

The Fundamental Data Structure: `board.cells`

The most critical attribute is `board.cells`.

- It is a Python **set**.
- It contains `(x, y)` coordinate tuples for all cells currently occupied by landed blocks on the grid.

Accessing the Game State

The Fundamental Data Structure: `board.cells`

The most critical attribute is `board.cells`.

- It is a Python **set**.
- It contains (x, y) coordinate tuples for all cells currently occupied by landed blocks on the grid.

Other Useful Attributes

You may also need information about the currently falling block (`board.falling`) and the next block (`board.next`).

Strategy Development

Strategy Development

The Algorithmic Challenge

How do we determine the optimal placement for the falling block?

The Core Problem

We need to evaluate potential landing positions and orientations for the current block and select the "best" one.

Connecting to Week 5: Search and Heuristics

This is fundamentally a search problem.

- **Search Space:** All possible landing positions and orientations.
- **Goal:** Maximise the score, or optimise the board state for future moves.

The Algorithmic Challenge

How do we determine the optimal placement for the falling block?

The Core Problem

We need to evaluate potential landing positions and orientations for the current block and select the "best" one.

Connecting to Week 5: Search and Heuristics

This is fundamentally a search problem.

- **Search Space:** All possible landing positions and orientations.
- **Goal:** Maximise the score, or optimise the board state for future moves.

The Algorithmic Challenge

Connecting to Week 5: Search and Heuristics

This is fundamentally a search problem.

- **Search Space:** All possible landing positions and orientations.
- **Goal:** Maximise the score, or optimise the board state for future moves.

The Need for Heuristics

Since we cannot know the entire future sequence of blocks, we cannot truly optimise (like Minimax in Chess). We must use **Heuristics** to evaluate the desirability of a board state.

Developing a Strategy: The Search Phase

A standard approach involves a systematic search (similar to DFS/BFS, but adapted for the Tetris environment).

The General Approach

- 1 **Explore Moves:** Iterate through all possible rotations and horizontal translations for the current block.
- 2 **Simulate Placement:** Determine where the block would land for each configuration.
- 3 **Evaluate:** Score the resulting board state using a heuristic function.
- 4 **Select:** Choose the move that leads to the highest-scoring state.

Simulation and The Sandbox

To evaluate moves without affecting the actual game state, you must use a sandbox.

The Sandbox (Week 5: Deepcopy)

Recall the importance of `deepcopy` in the Minimax implementation (Week 5) for exploring game trees. The same principle applies here.

Use `board.clone()` to create an independent copy of the game state for simulation.

Simulation Workflow

- 1 `sandbox = board.clone()`
- 2 Apply a sequence of moves (rotate/translate/drop) to the `sandbox`.
- 3 Analyse the resulting `sandbox.cells`.

Simulation and The Sandbox

To evaluate moves without affecting the actual game state, you must use a sandbox.

The Sandbox (Week 5: Deepcopy)

Recall the importance of `deepcopy` in the Minimax implementation (Week 5) for exploring game trees. The same principle applies here.

Use `board.clone()` to create an independent copy of the game state for simulation.

Simulation Workflow

- 1 `sandbox = board.clone()`
- 2 Apply a sequence of moves (rotate/translate/drop) to the `sandbox`.
- 3 Analyse the resulting `sandbox.cells`.

Developing a Strategy: The Heuristic Function

The most critical part of your AI is the Heuristic Evaluation Function. How do you define a "good" board state?

Common Heuristic Factors

Height: Lower aggregate height is generally better (penalty for tall stacks).

Holes: Penalise holes (empty spaces below landed blocks). Holes are difficult to fill.

Bumpiness: Penalise large height differences between adjacent columns (smooth surfaces are better).

Line Clears: Reward configurations that clear lines, especially multi-line clears (the primary objective).

Developing a Strategy: The Heuristic Function

Common Heuristic Factors

Height: Lower aggregate height is generally better (penalty for tall stacks).

Holes: Penalise holes (empty spaces below landed blocks). Holes are difficult to fill.

Bumpiness: Penalise large height differences between adjacent columns (smooth surfaces are better).

Line Clears: Reward configurations that clear lines, especially multi-line clears (the primary objective).

The Strategy

Your AI will likely combine these factors using weighted linear combinations. Finding the optimal weights is also a main challenge.

Advanced Considerations (Lookahead)

The basic strategy only considers the current block.

Bounded Depth Search (Week 5)

A more advanced AI might look ahead by considering the next block (`board.next`) as well.

- Evaluate the best placement for the current block, assuming the best placement for the next block follows.

The Complexity Trade-off (Week 3 & 4)

Increasing the search depth (lookahead) increases the time complexity exponentially ($\mathcal{O}(b^d)$). Given the autograder constraints, deep searches are likely infeasible.

Advanced Considerations (Lookahead)

The basic strategy only considers the current block.

Bounded Depth Search (Week 5)

A more advanced AI might look ahead by considering the next block (`board.next`) as well.

- Evaluate the best placement for the current block, assuming the best placement for the next block follows.

The Complexity Trade-off (Week 3 & 4)

Increasing the search depth (lookahead) increases the time complexity exponentially ($\mathcal{O}(b^d)$). Given the autograder constraints, deep searches are likely infeasible.

Returning Actions: Lists vs Generators

Once your AI has determined the best sequence of moves (e.g., Rotate, Left, Left, Drop), it must return them from `choose_action()`.

Option 1: Return a List

Return the entire sequence as a list:

```
return [Rotation.Clockwise, Direction.Left, Direction.Drop]
```


Returning Actions: Lists vs Generators

Option 2: Use a Generator (Week 4: `yield`)

You can use the `yield` keyword to return moves one by one.

- This can lead to cleaner code when generating the move sequence algorithmically.
- Recall Week 4: Generators provide lazy evaluation.

```
1 yield Rotation.Clockwise
2 yield Direction.Left
3 yield Direction.Drop
```

Note

The game engine will execute the returned moves until the block lands, then call `choose_action()` again for the next block.

Returning Actions: Lists vs Generators

Option 2: Use a Generator (Week 4: `yield`)

You can use the `yield` keyword to return moves one by one.

- This can lead to cleaner code when generating the move sequence algorithmically.
- Recall Week 4: Generators provide lazy evaluation.

```
1 yield Rotation.Clockwise
2 yield Direction.Left
3 yield Direction.Drop
```

Note

The game engine will execute the returned moves until the block lands, then call `choose_action()` again for the next block.

Conclusion and Advice

Conclusion and Advice

Summary of Key Steps

The Workflow

- 1 Fork and clone the repository.
- 2 Understand the codebase, especially `board.py` and `player.py`.
- 3 Implement your AI class and update `selected_player`.
- 4 Develop a strategy: Search + Heuristics.
- 5 Test locally (use the Pygame interface for speed).
- 6 Commit and push to the autograder.

Pedagogical Advice

Start by Understanding the Code Base

Before designing any complex strategy or heuristic, you must first understand the information available to you.

The Process

- 1 Inspect the board class.
- 2 Understand `board.cells`, `board.falling`, `board.next`.
- 3 Determine what information (Height, Holes, etc.) you can derive from this data.

The Principle

Strategy development must follow data-state comprehension.

Pedagogical Advice

Start by Understanding the Code Base

Before designing any complex strategy or heuristic, you must first understand the information available to you.

The Process

- 1 Inspect the board class.
- 2 Understand `board.cells`, `board.falling`, `board.next`.
- 3 Determine what information (Height, Holes, etc.) you can derive from this data.

The Principle

Strategy development must follow data-state comprehension.

Pedagogical Advice

Start by Understanding the Code Base

Before designing any complex strategy or heuristic, you must first understand the information available to you.

The Process

- 1 Inspect the board class.
- 2 Understand `board.cells`, `board.falling`, `board.next`.
- 3 Determine what information (Height, Holes, etc.) you can derive from this data.

The Principle

Strategy development must follow data-state comprehension.

Final Reminders

Key Considerations

- **Scoring:** Heavily prioritise multi-row clears (1600 points for 4 rows).
- **Constraints:** Be mindful of the memory (256MB) and time limits. Efficiency matters.
- **New Features:** Ignore Bombs and Discards initially.
- **Autograder:** Use the median score; push strategically to avoid queues.

Good Luck!